

Evaluation synthetischer Tabellendaten unter Low-Data-Bedingungen

Jack Gilbert

Department Informatik, HAW Hamburg
jack.gilbert@haw-hamburg.de

Zusammenfassung. Diese Arbeit untersucht in einer lokal ausgeführten Machbarkeitsstudie, wie sich synthetische tabellarische Daten unter Low-Data-Bedingungen erzeugen und bewerten lassen. Verglichen werden das Conditional Tabular Generative Adversarial Network (CTGAN), direkte In-Context-Learning-Zeilengenerierung (Direct-ICL) und Code-LLM, eine eingeschränkte Pipeline, die ein lokal betriebenes großes Sprachmodell (Large Language Model, LLM) zur Codegenerierung nutzt. Die Experimente erfolgen auf Adult und Pima mit 100 bzw. 500 Trainingszeilen, drei Zufallsstarts und jeweils 500 finalen synthetischen Zeilen. Bewertet werden Nutzbarkeit im Train-on-Synthetic-Test-on-Real-Setting (TSTR), Verteilungsnähe über eine zusammengesetzte Fidelity-Distanz, formale Gültigkeit, Zeilenherkunft, Fallback-Anteil und Laufzeit. Die drei Hauptmethoden bilden den Hauptvergleich; Fallback-only, CTGAN-strong und Direct-ICL Primary-only dienen als diagnostische Zusatzvarianten. Ergänzend werden einfache Datenschutzindikatoren, die Demographic Parity Difference (DPD) für Adult und eine semantische Plausibilitätsprüfung berichtet.

Alle 60 Läufe lieferten formal gültige finale Tabellen ohne exakte Trainingsduplikate. Im Hauptvergleich erzielte Direct-ICL in allen vier Datensatz-/Trainingsgrößen-Kombinationen den höchsten mittleren TSTR-F1. Auf Adult stützt die Primary-only-Auswertung diesen Befund, weil die primären Direct-ICL-Zeilen bereits hohe TSTR-F1-Werte erreichen. Auf Pima erreicht Fallback-only dagegen nahezu denselben TSTR-F1 wie Direct-ICL. Damit kann ein hoher TSTR-F1 ohne Zeilenherkunft irreführend sein und darf dort nicht automatisch als LLM-Leistung interpretiert werden. Code-LLM bleibt nahezu Fallback-frei und erzielt die niedrigsten zusammengesetzten Fidelity-Distanzen, erreicht aber keinen hohen TSTR-F1. CTGAN bleibt eine schnelle, Fallback-freie Baseline.

Schlüsselwörter: Synthetische Tabellendaten · Low-Data · Evaluation · Large Language Models · In-Context Learning

1 Einleitung und Einordnung

Synthetische tabellarische Daten können reale Daten ergänzen, wenn diese knapp, sensitiv oder schwer zugänglich sind [1,2]. Unter Low-Data-Bedingungen ist das attraktiv, aber riskant: Wenige Beispiele können instabile Verteilungen,

schwach erfasste Zusammenhänge oder eine ungewollte Nähe zu Trainingsdaten begünstigen [3]. Deshalb sollte die Evaluation neben formaler Gültigkeit auch nachgelagerte Nutzbarkeit, Verteilungsnähe, Zeilenherkunft sowie Datenschutzindikatoren und eine explorative Fairnessprüfung erfassen [4]. Frameworks wie Synthcity und SynthEval bündeln solche Perspektiven typischerweise in Nutzbarkeit, Verteilungsnähe und Risikodimensionen [5,6].

Diese Dimensionen fallen nicht notwendig zusammen: Eine Methode kann formal gültige Tabellen liefern, ohne aufgabenrelevante Merkmal-Zielvariable-Zusammenhänge zu reproduzieren, umgekehrt können gute Werte im Train-on-Synthetic-Test-on-Real-Setting (TSTR-Setting) durch Pipeline-Schritte mitgetragen werden. Als Fallback-Mechanismus bezeichnet diese Arbeit den Schritt, mit dem fehlende formal gültige synthetische Zeilen aus Trainingsstatistiken ergänzt werden. Er greift nur, wenn der primäre Generator nach Parsing, Normalisierung, Validierung und Deduplizierung weniger als 500 formal gültige Zeilen liefert. Die dadurch erzeugten Zeilen heißen Fallback-Zeilen, ihr Anteil an der finalen synthetischen Tabelle wird als Fallback-Anteil berichtet. Entscheidend ist daher, wie viele Zeilen als primäre Generatorzeilen und wie viele als Fallback-Zeilen in die finale Tabelle eingehen.

Die Studie ist artefaktbasiert nachvollziehbar und unter dokumentierter lokaler Ollama-/Modellumgebung wiederholbar. Die Reproduktion beginnt bei den vorverarbeiteten CSVs, archivierten Splits und Ergebnisartefakten, nicht bei einer Rohdatenpipeline, lokale LLM-Inferenz ist zudem nicht notwendigerweise bitgenau reproduzierbar. Zugleich bleibt die Arbeit eine Machbarkeitsstudie mit zwei Datensätzen, drei Zufallsstarts und einem lokalen 3B-Modell.

Aufbauend auf einer früheren theoretischen Ausarbeitung zu LLM-gestützter tabellarischer Datensynthese untersucht diese Arbeit eine praktische Versuchsanordnung [7]. Der Hauptvergleich umfasst das Conditional Tabular Generative Adversarial Network (CTGAN) als klassische Baseline, direkte In-Context-Learning-Zeilengenerierung (Direct-ICL) und Code-LLM als LLM-basierte Pipeline, die ein kleines Programm zur Datenerzeugung erstellt.

Die Zusatzvarianten Fallback-only, CTGAN-strong und Direct-ICL Primary-only prüfen zentrale Einflussfaktoren der Ergebnisse getrennt und werden nicht als weitere Kandidaten im Methodenvergleich gewertet. Fallback-only nutzt ausschließlich den Fallback-Mechanismus ohne primären Generator. CTGAN-strong bezeichnet CTGAN mit 300 statt 50 Epochen. Primary-only bezeichnet die Auswertung der formal gültigen primären Direct-ICL-Zeilen vor Fallback. Die Auswahl orientiert sich einerseits an klassischen Generatoren und LLM-basierter Tabellensynthese [1,2], andererseits an In-Context Learning und Codegenerierung [8,9].

Der praktische Beitrag ist eine artefaktbasiert nachvollziehbare Evaluationspipeline mit Fokus auf Zeilenherkunft. Die Studie trennt primäre Generatorzeilen und Fallback-Zeilen, damit hohe TSTR-F1-Werte nicht dem primären Generator allein zugeschrieben werden. Berichtet werden TSTR-F1, zusammengesetzte Fidelity-Distanz, formale Gültigkeit, Fallback-Anteil, Laufzeit sowie Datenschutzindikatoren, Adult-DPD und eine semantische Plausibilitätsprüfung.

Die Leitfrage lautet daher: Wie unterscheiden sich CTGAN als klassische Baseline, Direct-ICL und Code-LLM unter kontrollierten Low-Data-Bedingungen? Die Studie umfasst 36 Hauptläufe und 24 zusätzliche Läufe und zielt auf eine nachvollziehbare Bewertung der Pipeline-Bestandteile, nicht auf eine allgemeine Rangfolge.

2 Methodik

Alle Generatorpfade folgen derselben Grundlogik: Aus einem realen Trainingsausschnitt werden Kandidatenzeilen erzeugt, anschließend normalisiert, validiert, dedupliziert und bei Bedarf auf 500 finale Zeilen ergänzt. Die Auswertung trennt deshalb primäre Generatorzeilen und Fallback-Zeilen, bevor Nutzbarkeit, Verteilungsnähe, Integrität und Zusatzvarianten berichtet werden. Zunächst werden Datensätze, Splits und Design beschrieben, danach folgen Generatoren, gemeinsame Pipeline und Metriken.

2.1 Datensätze, Splits und Design

Das Experiment verwendet zwei vorverarbeitete Datensätze mit binären Zielvariablen. Adult ist gemischt numerisch-kategorial, umfasst 46.033 Zeilen und 10 Spalten mit der Zielvariable `income`. Pima ist ein kleines, überwiegend numerisches medizinisches Klassifikationsproblem mit 768 Zeilen, 9 Spalten und der Zielvariable `Outcome`. Bei Adult ist `sex` ein reguläres kategoriales Merkmal und Merkmal für die Demographic Parity Difference (DPD), für Pima wird kein sensibles Gruppenmerkmal ausgewertet. Beide Datensätze sind etablierte Referenzen für tabellarische Klassifikation [10,11]. Die Kombination erlaubt zu prüfen, ob Generator- und Fallback-Mechanismus je nach Datentyp unterschiedlich wirken.

Das experimentelle Setup umfasst Adult und Pima, Trainingsausschnitte mit 100 bzw. 500 Zeilen, drei Zufallsstarts und die Hauptmethoden CTGAN, Direct-ICL und Code-LLM, die vollständige Konfiguration steht in Tabelle 4 im Anhang. Jeder Lauf erzeugt 500 finale synthetische Zeilen. CTGAN wird in der Hauptvariante mit 50 Epochen trainiert, CTGAN-strong mit 300 Epochen. Direct-ICL nutzt CSV-Prompting mit Batchgröße 25 und Temperatur 0.5, Code-LLM ein festes Codegerüst mit Temperatur 0.1, niedrigere Temperaturen führen tendenziell zu deterministischeren LLM-Ausgaben.

Für beide Datensätze liegt ein vorverarbeiteter, gehashter CSV-Stand vor, die Paketreproduktion beginnt dort und bei den archivierten Splits, nicht bei einer Rohdaten-zu-processed-Pipeline. Zunächst wurde ein Testsplit von 20 stratifiziert mit `random_state=0` abgetrennt. Die Trainingsausschnitte mit 100 bzw. 500 Zeilen werden stratifiziert aus dem verbleibenden Trainingspool mit den Seeds 42, 123 und 2025 gezogen. Innerhalb jeder Kombination nutzen alle Methoden denselben Trainingsausschnitt und Testsplit.

Das Hauptdesign umfasst CTGAN, Direct-ICL und Code-LLM auf jeder Datensatz-/Trainingsgrößen-/Zufallsstart-Kombination, also 36 Hauptläufe. Fallback-only und CTGAN-strong laufen ebenfalls auf allen zwölf Kombinationen

und ergeben zusammen 24 zusätzliche Läufe. Primary-only wird dagegen aus den Direct-ICL-Läufen abgeleitet und erhöht die Laufzahl nicht. Jeder Lauf erzeugt 500 finale synthetische Zeilen. Zur Nachvollziehbarkeit archiviert das Reproduzierbarkeitspaket Splits, Konfigurationen, Ergebnisdateien, synthetische Ausgaben, Zeilenherkunft und Zusatztabellen. Alle berichteten Ergebnisse stammen aus diesem Paket mit der in Tabelle 4 angegebenen Konfiguration [12].

2.2 Generatoren und Pipeline

CTGAN trainiert auf dem jeweiligen Trainingsausschnitt und erzeugt Kandidatenzeilen, die dieselbe Validierungs- und Deduplizierungspipeline wie die anderen Methoden durchlaufen. Die Hauptexperimente verwenden den CTGAN-Synthesizer aus SDV mit 50 Epochen und sonstigen Default-Parametern, im finalen Lauf wurden `sdv==1.37.0` und `ctgan==0.12.1` verwendet [13]. Pro Lauf werden Kandidaten nachgezogen, bis 500 gültige Zeilen vorliegen oder höchstens das 20-Fache der Zielgröße gesampelt wurde. CTGAN-strong nutzt 300 Epochen und ordnet diese kompakte CTGAN-Baseline zusätzlich ein.

Für Adult wurde `education-num` bei CTGAN nach dem Sampling deterministisch aus `education` rekonstruiert, statt als unabhängige Ausgabespalte modelliert zu werden, validiert wurde anschließend im vollständigen Zielschema.

Direct-ICL erzeugt CSV-Zeilen in Batches von 25 und mit höchstens 40 Batch-Anfragen. Jeder Prompt enthält acht seed-kontrolliert aus dem jeweiligen Trainingsausschnitt stratifiziert gezogene Beispielzeilen, die Spaltenreihenfolge, die empirische Zielverteilung und die Anweisung, Trainingsbeispiele nicht exakt zu kopieren [8]. Als Primary-only-Auswertung werden die formal gültigen primären Direct-ICL-Zeilen vor Fallback mit TSTR-F1 und zusammengesetzter Fidelity-Distanz bewertet. Primary-only nutzt höchstens 500 solcher Zeilen; wegen der teilweise geringeren Zeilenzahl, besonders auf Adult, ist dies nicht als zusätzliche gleichartige 500-Zeilen-Generierungsmethode zu lesen.

Code-LLM ist keine frei programmierende Methode, sondern eine stark gerahmte Pipeline zur Codegenerierung. Das Modell erzeugt keinen beliebigen Code, sondern nur den Rumpf der Funktion `generate_synthetic_data(n, seed)` innerhalb eines festen Gerüsts. Spaltenreihenfolge, Schema, erlaubte Kategorien, Wertebereiche, Zufallszahlengenerator und wenige Sampling-Hilfsfunktionen sind vorgegeben. Dem Prompt werden keine vollständigen Trainingszeilen übergeben, sondern Aggregatstatistiken: für numerische Merkmale Minimum, Maximum, Mittelwert, Standardabweichung und Quantile, für kategoriale Merkmale und Zielvariable geglättete empirische Häufigkeiten.

Der erzeugte Funktionsrumpf wird statisch auf verbotene Konstrukte wie Imports, Dateioperationen, dynamische Codeausführung und nicht vorgesehene Hilfsfunktionen geprüft, kompiliert und in einem Smoke-Test ausgeführt. Erst danach wird der Code lokal zur Kandidatenerzeugung genutzt; bis zu fünf LLM- bzw. Reparaturversuche sind vorgesehen [9]. Dies begrenzt den Funktionsrumpf praktisch, ist aber keine formale Vollständigkeitsgarantie.

Alle Methoden durchlaufen dieselbe Pipeline: Rohausgaben werden ins Zielschema gebracht, auf Vollständigkeit, erlaubte Kategorien, Wertebereiche, Ziel-

werte und formale Nebenbedingungen geprüft und von internen sowie exakten Trainingsduplikaten bereinigt. Dadurch gelten für alle finalen synthetischen Tabellen dieselben schema- und wertebezogenen Mindestanforderungen.

Wenn nach Parsing, Normalisierung, Validierung und Deduplizierung weniger als 500 formal gültige Zeilen vorliegen, ergänzt der Fallback-Mechanismus die Tabelle reproduzierbar aus Trainingsstatistiken. Kategoriale Werte werden aus Laplace-geglätteten Häufigkeiten gezogen, numerische Werte entstehen aus Stichproben der 5-, 25-, 50-, 75- und 95-Prozent-Quantile mit kleinem Jitter und werden auf die erlaubten Wertebereiche begrenzt. Für Adult nutzt der Fallback zusätzlich die `education-zu-education-num`-Abbildung und eine `workclass`-bedingte `occupation`-Auswahl. Für Pima wird `Outcome` aus den geglätteten Trainingshäufigkeiten gezogen, numerische Merkmale werden aus `Outcome`-spezifischen Trainingsquantilen erzeugt, falls die jeweilige Zielklasse mindestens fünf Trainingszeilen enthält, sonst aus dem gesamten Trainingsausschnitt. Fallback-Zeilen werden erneut validiert, dedupliziert und über die Zeilenherkunft von primären Generatorzeilen getrennt. Fallback-only nutzt ausschließlich diesen Mechanismus und zeigt, welcher TSTR-F1 bereits ohne primären Generator erreichbar ist [12].

2.3 Metriken und Interpretationsgrenzen

Die Evaluation kombiniert mehrere Metriken, da formale Gültigkeit allein weder Nutzbarkeit noch Verteilungsnähe, Zeilenherkunft oder Risiken abbildet.

Zur Berechnung des TSTR-F1 wird eine feste `sklearn`-Pipeline auf synthetischen Daten trainiert und auf dem realen Testsplit evaluiert [14,15,16]. Numerische Merkmale werden mit `StandardScaler` skaliert, kategoriale Merkmale mit `OneHotEncoder(handle_unknown=ignore)` kodiert; anschließend wird `LogisticRegression(max_iter=1000, random_state=0)` verwendet. Berichtet wird der F1-Wert der positiven Zielklasse, also `income >50K` für Adult und `Outcome=1` für Pima, über drei Zufallsstarts. Der Real-Train-F1 bei Training auf dem entsprechenden realen Trainingsausschnitt dient als Real-Train-Referenz. TSTR-F1 als Nutzbarkeitsmaß ist an diese Logistic-Regression-Pipeline gebunden, andere Downstream-Modelle könnten andere Rangfolgen ergeben.

Die zusammengesetzte Fidelity-Distanz misst die Verteilungsnähe zwischen synthetischen und realen Trainingsdaten. Sie ist definiert als

$$D_{\text{fid}} = \text{nanmean}(D_{\text{num}}, D_{\text{cat}}, D_{\text{target}}, D_{\text{corr}}).$$

Dabei ist D_{num} der Mittelwert der Kolmogorov-Smirnov-Abstände über nicht-zielvariable numerische Merkmale, D_{cat} der Mittelwert der Total-Variation-Abstände über nicht-zielvariable kategoriale Merkmale, D_{target} der Total-Variation-Abstand der Zielverteilung und D_{corr} die mittlere absolute Abweichung der numerischen Korrelationsmatrizen. Nicht verfügbare Komponenten werden in der Mittelung ignoriert, die Gleichgewichtung ist eine pragmatische Designentscheidung und keine standardisierte Qualitätsmetrik. Zielklassenkontingierten Abstände und Merkmal-Zielvariable-Mittelwertabweichungen sind im Paket dokumentiert, gehen aber nicht in D_{fid} ein. Niedrigere Werte bedeuten größere Verteilungsnähe nach diesen Distanzmaßen.

Die Exact-Train-Duplicate-Rate bezeichnet den Anteil finaler synthetischer Zeilen, die nach Normalisierung exakte Duplikate des Trainingsausschnitts sind. Sie ist ein enger Datenschutzindikator: Ein Wert von null zeigt nur, dass keine offensichtlichen exakten Kopien in den finalen synthetischen Tabellen vorliegen; Membership-Inference-Analysen, also Prüfungen, ob sich die Zugehörigkeit einer realen Zeile zum Trainingsset erkennen lässt, oder Garantien nach Differential Privacy, einem formalen Datenschutzrahmen mit quantifizierbarem Schutzversprechen, ersetzt er nicht [3,17]. Für Adult ergänzt die Evaluation die mittlere absolute Adult-DPD, also die absolute Differenz der positiven Zielklassenanteile zwischen `sex`-Gruppen, für die positive Klasse `>50K` der Zielvariable `income`. Die absoluten DPD-Werte werden im Vergleich zur Adult-DPD der realen Trainingsdaten eingeordnet. Adult-DPD dient hier als Fairnessindikator, nicht als belastbare Fairnessbewertung [18]. Die Laufzeit wird bei jedem Lauf lokal gemessen.

Ergänzend prüft eine regelbasierte semantische Plausibilitätsprüfung auffällige Merkmalskombinationen. Die Regeln sind datensatznahe Warnindikatoren, keine harten Fehlerlabels. Die Werte wurden post-hoc aus den archivierten finalen synthetischen CSVs berechnet und über Trainingsgrößen und Seeds gepoolt. Reale Vergleichswerte stammen aus den archivierten Trainingsausschnitten, nur bei ungesesehenen Adult-`workclass/occupation`-Paaren dient die Menge der im vorverarbeiteten Adult-Gesamtdatensatz beobachteten Paare als Referenz.

3 Ergebnisse

Die Ergebnisse werden aus fünf Perspektiven dargestellt: Zuerst stehen Integrität der finalen Tabellen und Zeilenherkunft im Vordergrund, danach TSTR-F1, zusammengesetzte Fidelity-Distanz und Laufzeit der Hauptmethoden. Anschließend ordnen Zusatzvarianten und datensatzspezifische Muster die wichtigsten Befunde ein, zuletzt folgen Datenschutzindikatoren, Adult-DPD und semantische Plausibilitätsprüfung.

3.1 Integrität der finalen Tabellen und Zeilenherkunft

Alle 60 Läufe der vollständigen Ergebnismatrix, also 36 Hauptläufe und 24 zusätzliche Läufe, schlossen erfolgreich ab. Jede finale synthetische Tabelle enthielt 500 Zeilen, bestand die Gültigkeitschecks und enthielt keine exakten Trainingsduplikate. Dieser Befund betrifft die finalen synthetischen Tabellen nach Validierung, Deduplizierung und gegebenenfalls Fallback, nicht die Rohqualität der Generatoren.

Die Zeilenherkunft unterscheidet sich klar. CTGAN und CTGAN-strong waren Fallback-frei. Direct-ICL lieferte in allen 12 Läufen formal gültige finale synthetische Tabellen, erforderte auf Adult aber einen deutlichen Fallback-Anteil. Über alle 12 Direct-ICL-Läufe hinweg entstanden 1244 Fallback-Zeilen; die in Tabelle 1 berichteten Fallback-Anteile zeigen, dass der Schwerpunkt dabei auf Adult liegt. Code-LLM war in allen 12 Läufen technisch erfolgreich, davon elf Fallback-frei, ein Adult-500-Lauf enthielt eine einzelne Fallback-Zeile.

3.2 TSTR-F1, zusammengesetzte Fidelity-Distanz und Laufzeit

Tabelle 1 bildet den Hauptvergleich zwischen CTGAN, Direct-ICL und Code-LLM ab. TSTR-F1 dient als Nutzbarkeitsmaß, die zusammengesetzte Fidelity-Distanz die Verteilungsnähe, die Laufzeit den praktischen Aufwand und der Fallback-Anteil den Anteil der Fallback-Zeilen.

Tabelle 1. Hauptvergleich der drei Hauptmethoden. $\uparrow$ bedeutet höher ist besser, $\downarrow$ bedeutet niedriger ist besser; FB = Fallback-Anteil.

Daten	Methode	n	TSTR-F1 $\uparrow$	Fid. $\downarrow$	Zeit (s) $\downarrow$	FB $\downarrow$
Adult	CTGAN	100	0.030 ± 0.012	0.127 ± 0.004	1.9 ± 0.1	0.000
Adult	Direct-ICL	100	0.466 ± 0.046	0.137 ± 0.012	569.2 ± 68.8	0.385
Adult	Code-LLM	100	0.049 ± 0.046	0.075 ± 0.010	16.8 ± 1.3	0.000
Adult	CTGAN	500	0.018 ± 0.013	0.127 ± 0.006	7.2 ± 0.3	0.000
Adult	Direct-ICL	500	0.518 ± 0.063	0.117 ± 0.020	671.7 ± 24.6	0.435
Adult	Code-LLM	500	0.074 ± 0.049	0.064 ± 0.005	18.1 ± 0.8	0.001
Pima	CTGAN	100	0.415 ± 0.133	0.196 ± 0.019	2.0 ± 0.1	0.000
Pima	Direct-ICL	100	0.494 ± 0.044	0.150 ± 0.007	492.6 ± 3.2	0.001
Pima	Code-LLM	100	0.076 ± 0.081	0.102 ± 0.009	16.1 ± 0.7	0.000
Pima	CTGAN	500	0.281 ± 0.170	0.205 ± 0.019	7.4 ± 0.0	0.000
Pima	Direct-ICL	500	0.495 ± 0.006	0.136 ± 0.013	492.1 ± 4.8	0.008
Pima	Code-LLM	500	0.057 ± 0.069	0.098 ± 0.004	15.7 ± 0.1	0.000

Werte sind Mittelwert $\pm$ Standardabweichung über drei Zufallsstarts. n = Trainingszeilen; Fid. = zusammengesetzte Fidelity-Distanz.

Über alle vier Datensatz-/Trainingsgrößen-Kombinationen hinweg zeigt sich ein klarer Trade-off: Direct-ICL erreicht den höchsten TSTR-F1, ist aber mit Abstand am langsamsten. Code-LLM erzielt die niedrigste zusammengesetzte Fidelity-Distanz, reproduziert jedoch die für den Klassifikator relevanten Zielzusammenhänge nur schwach; CTGAN bleibt die schnellste fallback-freie Baseline.

Zur Einordnung der TSTR-F1-Werte liegt der Real-Train-F1 bei Adult bei 0.548 bzw. 0.591 für $n = 100/500$ und bei Pima bei 0.493 bzw. 0.500. Direct-ICL liegt damit auf Pima nahe an dieser Real-Train-Referenz, auf Adult aber darunter.

Tabelle 2 ordnet die TSTR-F1-Werte der Zusatzvarianten ein. Primary-only ist eine Auswertung der verfügbaren primären Direct-ICL-Zeilen und wegen der teils geringeren Zeilenzahlen nicht als vollständig gleichartige 500-Zeilen-Methode zu lesen. Fallback-only zeigt, was der Fallback-Mechanismus allein leistet, und CTGAN-strong prüft, ob längeres Training die kompakte CTGAN-Baseline verändert. Die datensatzspezifische Interpretation folgt in Abschnitt 3.3.

Tabelle 2. TSTR-F1 der Zusatzvarianten.

Daten	n	Direct-ICL final	Primary-only	Fallback-only	CTGAN	CTGAN-strong
Adult	100	0.466	0.529	0.011	0.030	0.111
Adult	500	0.518	0.530	0.033	0.018	0.055
Pima	100	0.494	0.499	0.487	0.415	0.301
Pima	500	0.495	0.481	0.491	0.281	0.381

n = Trainingszeilen; deskriptive Mittelwerte über drei Zufallsstarts. Primary-only nutzt im Mittel 307/282 Zeilen für Adult mit $n = 100/500$ und 500/496 Zeilen für Pima mit $n = 100/500$.

Die Laufzeiten unterscheiden sich deutlich: CTGAN ist am schnellsten, über alle Datensatz-/Trainingsgrößen-Kombinationen gemittelt liegt Code-LLM mit etwa 16,7 Sekunden klar vor Direct-ICL, das rund 556 Sekunden benötigt. Der nächste Abschnitt interpretiert diese Werte datensatzspezifisch mithilfe von Primary-only und Fallback-only. Adult und Pima werden getrennt betrachtet, da Adult gemischt numerisch-kategorial ist, während Pima überwiegend numerische Merkmale enthält.

3.3 Datensatzspezifische Muster

Die wichtigsten Unterschiede liegen in der Herkunft der Direct-ICL-Werte. Auf Adult erreicht Direct-ICL final deutlich höhere TSTR-F1-Werte als CTGAN und Code-LLM, bleibt aber unter der Real-Train-Referenz und weist einen deutlichen Fallback-Anteil auf. Primary-only ist dort stärker, nutzt aber nur die verfügbaren primären Zeilen vor Fallback; Fallback-only ist sehr schwach. Das spricht dafür, dass die primären Direct-ICL-Zeilen selbst nutzbare Strukturen enthalten, während der Fallback-Mechanismus vor allem die Zielgröße von 500 Zeilen sicherstellt.

Auf Pima liegt der zentrale Befund darin, dass Direct-ICL fast Fallback-frei ist, Fallback-only aber nahezu denselben TSTR-F1 erreicht. Ohne Zeilenherkunft wäre der hohe TSTR-F1 daher irreführend, weil er nicht automatisch als LLM-Leistung interpretiert werden darf, einfache Outcome-spezifische Trainingsstatistiken scheinen für die verwendete Logistic Regression bereits viel relevante Struktur zu erfassen.

Code-LLM erzielt niedrige zusammengesetzte Fidelity-Distanzen, aber keinen hohen TSTR-F1. Da der Prompt vor allem marginale Aggregatstatistiken und keine expliziten Merkmal-Zielvariable-Beziehungen enthält, zeigt der Befund eine Grenze der verwendeten zusammengesetzten Fidelity-Distanz: Einzelverteilungen, Zielhäufigkeiten und numerische Korrelationen können gut approximiert werden, ohne ausreichend prädiktive Strukturen für den gewählten Klassifikator zu reproduzieren. Entscheidend sind daher auch die für den Klassifikator relevanten Zusammenhänge. Die niedrigen Adult-TSTR-F1-Werte deuten darauf hin, dass CTGAN in dieser kompakten Konfiguration solche Zusammenhänge nur unzureichend reproduziert. CTGAN-strong verbessert einzelne Konstellationen, ändert das Gesamtmuster aber nicht konsistent.

3.4 Datenschutzindikatoren und explorative Fairnessprüfung

In allen 60 finalen synthetischen Tabellen liegt die Exact-Train-Duplicate-Rate bei null, exakte Trainingsduplikate traten also nicht auf. Der Befund ist ein Plausibilitätscheck gegen offensichtliches Auswendiglernen. Er schließt jedoch sehr ähnliche, aber nicht exakt gleiche Kopien (Near-Duplicates), Membership-Inference-Risiken und Risiken einer näherungsweisen Rekonstruktion realer Trainingsdaten nicht aus, eine Garantie nach Differential Privacy entsteht daraus ebenfalls nicht [3,17].

Für Adult zeigt Tabelle 3 zusätzlich die mittlere absolute Adult-DPD für **sex** und die positive Klasse **income >50K**. Direct-ICL weist unter den Hauptmethoden die höchsten Adult-DPD-Werte auf, bleibt aber unter der Adult-DPD der realen Trainingsdaten. Niedrigere Adult-DPD-Werte können auch abgeschwächte Gruppenunterschiede anzeigen und sind hier keine Fairnessbewertung [18]. Dafür wären mehr Zufallsstarts, weitere Fairnessmetriken und eine genauere Analyse der Gruppenverteilungen notwendig.

Tabelle 3. Referenz, Datenschutzindikator und Adult-DPD.

Daten	n	Real-Train-F1	Dup. max	Adult-DPD			
				Real	CTGAN	D-ICL	C-LLM
Adult	100	0.548 ± 0.009	0.000	0.207 ± 0.071	0.027	0.117	0.037
Adult	500	0.591 ± 0.023	0.000	0.204 ± 0.011	0.050	0.079	0.041
Pima	100	0.493 ± 0.008	0.000	–	–	–	–
Pima	500	0.500 ± 0.009	0.000	–	–	–	–

Dup. max = maximale Exact-Train-Duplicate-Rate; Adult-DPD wird nur für Adult berichtet. D-ICL = Direct-ICL; C-LLM = Code-LLM.

3.5 Semantische Plausibilitätsprüfung

Die semantische Plausibilitätsprüfung ergänzt die bisherigen Metriken um post-hoc berechnete Warnindikatoren; Regeln und Werte stehen in Tabelle 5. Die Prüfung zeigt, dass formale Gültigkeit nicht automatisch semantische Plausibilität einzelner Zeilen impliziert. Besonders auffällig ist, dass CTGAN bei Adult junge verheiratete Personen deutlich häufiger erzeugt als die realen Trainingsausschnitte (11.30 % gegenüber 0.44 %), während Direct-ICL mit 3.77 % näher an der Realverteilung bleibt. Auf Pima hat Code-LLM bei niedrigen positiven Blutdruckwerten den höchsten Warnanteil (10.00 % gegenüber 0.17 % real), auch die Kombination aus mehr als acht Schwangerschaften und einem Alter unter 30 Jahren ist bei Code-LLM und Fallback-only auffällig (8.60 % bzw. 8.40 % gegenüber 0.22 % real). Damit ergänzt die Prüfung TSTR-F1 und zusammengesetzte Fidelity-Distanz um eine zeilenbezogene Qualitätsdimension, hohe Werte bleiben Hinweise auf schwach modellierte Abhängigkeiten und keine harten Fehler.

4 Diskussion und Limitationen

Die Ergebnisse zeigen, dass Nutzbarkeit, Verteilungsnähe, Laufzeit und Zeilenherkunft im geprüften Setting nicht zusammenfallen. Direct-ICL erreicht den höchsten mittleren TSTR-F1, Code-LLM die niedrigsten zusammengesetzten Fidelity-Distanzen und CTGAN die kürzeste Laufzeit, interpretierbar sind diese Befunde aber nur mit der Zeilenherkunft. Auf Adult sprechen die Primary-only-Werte für nutzbare primäre Direct-ICL-Zeilen, während auf Pima Fallback-only nahezu denselben TSTR-F1 wie Direct-ICL erreicht. Zugleich zeigt die Plausibilitätsprüfung, dass formale Gültigkeit und günstige Metrikwerte keine realistisch wirkenden Einzelzeilen garantieren.

Die TSTR-Befunde sind modellgebunden, auch die zusammengesetzte Fidelity-Distanz ist eine pragmatisch gewichtete Zusammenfassung.

Praktisch dominiert keine der drei Hauptmethoden allein. Steht nachgelagerte Nutzbarkeit im Vordergrund, ist Direct-ICL im geprüften Setting am stärksten, muss aber wegen Fallback-Anteil und Laufzeit kontrolliert werden. Für schnelle formal gültige Generierung bleibt CTGAN attraktiv, reproduziert hier aber prädiktive Zusammenhänge nur begrenzt. Code-LLM ist als kontrollierte lokale Pipeline interessant, benötigt aber stärkere Mechanismen für Merkmal-Zielvariable-Zusammenhänge.

Die Aussagekraft ist auf zwei Datensätze, zwei Trainingsgrößen, drei Zufallsstarts, ein lokales 3B-Modell und einen festen logistischen Klassifikator begrenzt; Exact-Train-Duplicate-Rate und Adult-DPD bleiben einfache Indikatoren. Wegen nur drei Zufallsstarts sind Mittelwerte und Standardabweichungen explorativ und nicht als robuste Signifikanzbewertung zu lesen. Die Ergebnisse zeigen daher Muster innerhalb des geprüften Artefakts, aber keine abgesicherte Überlegenheit über Datensätze, Modellgrößen oder Downstream-Aufgaben hinweg.

5 Fazit

Die Arbeit zeigt, dass synthetische tabellarische Daten unter Low-Data-Bedingungen artefaktbasiert nachvollziehbar bewertet werden können, wenn TSTR-F1, zusammengesetzte Fidelity-Distanz, Zeilenherkunft, Laufzeit und Fallback-Anteil gemeinsam betrachtet werden. Direct-ICL erreicht den höchsten mittleren TSTR-F1, Code-LLM die niedrigsten zusammengesetzten Fidelity-Distanzen und CTGAN die kürzeste Laufzeit.

Die Zusatzvarianten zeigen zugleich, dass hohe TSTR-F1-Werte nur mit Zeilenherkunft belastbar interpretierbar sind: Auf Adult stützt Primary-only den Direct-ICL-Befund, während Fallback-only auf Pima nahezu denselben TSTR-F1 wie Direct-ICL erreicht. Der Beitrag liegt daher in einem transparenten Bewertungsrahmen, der Pipeline-Effekte und Generatorleistung trennt. Die ergänzenden Datenschutzindikatoren, Adult-DPD-Werte und Plausibilitätsprüfungen zeigen wichtige Grenzen, ersetzen aber keine vertiefte Risiko- oder Domänenbewertung. Künftige Arbeiten sollten den Rahmen auf weitere Datensätze, Generatoren, lokale LLMs, Zufallsstarts und Downstream-Modelle übertragen und die Risikoanalysen vertiefen.

A Zusätzliche Tabellen

Tabelle 4. Setup der archivierten Ergebnisversion und Abgrenzung von Hauptvergleich und Zusatzvarianten.

Bereich	Dimension	Wert
<i>Design</i>	Artefakt-/Lauf-ID	20260629T202543Z
	Datensätze	Adult, Pima
	Trainingszeilen	100, 500
	Zufallsstarts (Seeds)	42, 123, 2025
	Hauptmethoden	CTGAN, Direct-ICL, Code-LLM
	Zusatzvarianten	Fallback-only, CTGAN-strong; Primary-only als abgeleitete Auswertung aus Direct-ICL-Läufen
	Finale Zeilen je Lauf Experimente	500 synthetische Zeilen 36 Hauptläufe + 24 zusätzliche Läufe = 60; Primary-only ist keine zusätzliche Laufklasse
<i>Konfiguration</i>	LLM-Inferenz	lokal über Ollama, 11ama3.2:3b
	CTGAN	50 Epochen
	CTGAN-strong	300 Epochen statt 50
	Direct-ICL	CSV-Prompting (comma-separated values), Batchgröße 25, Temperatur 0.5
	Code-LLM	festes Codegerüst, Temperatur 0.1

Tabelle 5. Ausgewählte semantische Plausibilitätsprüfung in Prozent.

Daten	Auffälligkeit	Real	CTGAN	D-ICL	C-LLM	FB-only
Adult	age < 22 und verheiratet	0.44	11.30	3.77	6.43	6.57
Adult	age < 25 und income > 50K	0.22	6.43	3.57	4.73	5.13
Adult	ungesehene workclass/occupation -Paare	0.00	0.37	0.93	0.90	2.60
Pima	Pregnancies > 8 und Age < 30	0.22	5.50	4.20	8.60	8.40
Pima	BloodPressure zwischen 1 und 39	0.17	5.83	0.10	10.00	6.93
Pima	positiver BMI < 18.5	0.56	6.10	0.03	0.77	2.10

Die Regeln sind Warnindikatoren, keine harten Fehlerdefinitionen. Werte wurden post-hoc aus finalen synthetischen CSVs berechnet und über Trainingsgrößen und Seeds gepoolt. Für **ungesehene workclass/occupation**-Paare dient der vorverarbeitete Adult-Gesamtdatensatz als Referenz.

Literatur

1. Xu, L., Skoularidou, M., Cuesta-Infante, A., Veeramachaneni, K.: Modeling tabular data using conditional GAN. In: Advances in Neural Information Processing Systems 32 (NeurIPS 2019) (2019)

2. Borisov, V., Sekler, K., Leemann, T., Pawelczyk, M., Kasneci, G.: Language models are realistic tabular data generators. In: International Conference on Learning Representations (ICLR 2023) (2023)
3. van Breugel, B., Sun, H., Qian, Z., van der Schaar, M.: Membership inference attacks against synthetic data through overfitting detection. In: Proc. 26th Int. Conf. on Artificial Intelligence and Statistics (AISTATS 2023). PMLR 206, 3493–3514 (2023)
4. van Breugel, B., Qian, Z., van der Schaar, M.: Synthetic data, real errors: how (not) to publish and use synthetic data. In: Proc. 40th Int. Conf. on Machine Learning (ICML 2023). PMLR 202, 34793–34808 (2023)
5. Qian, Z., Davis, R., van der Schaar, M.: Synthcity: a benchmark framework for diverse use cases of tabular synthetic data. In: Advances in Neural Information Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track, 3173–3188 (2023)
6. Lautrup, A.D., Hyrup, T., Zimek, A., Schneider-Kamp, P.: SynthEval: A framework for detailed utility and privacy evaluation of tabular synthetic data. arXiv preprint arXiv:2404.15821 (2024)
7. Gilbert, J.: LLM-basierte Generierung synthetischer tabellarischer Daten. Ausarbeitung, Department Informatik, HAW Hamburg (2025)
8. Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al.: Language models are few-shot learners. In: Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 1877–1901 (2020)
9. Nijkamp, E., Pang, B., Hayashi, H., Tu, L., Wang, H., Zhou, Y., Savarese, S., Xiong, C.: CodeGen: an open large language model for code with multi-turn program synthesis. In: International Conference on Learning Representations (ICLR 2023) (2023)
10. Kohavi, R.: Scaling up the accuracy of naive-Bayes classifiers: a decision-tree hybrid. In: Proc. Second Int. Conf. on Knowledge Discovery and Data Mining (KDD 1996), 202–207 (1996)
11. Smith, J.W., Everhart, J.E., Dickson, W.C., Knowler, W.C., Johannes, R.S.: Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. In: Proc. Annual Symposium on Computer Application in Medical Care, 261–265 (1988)
12. Gilbert, J.: Reproduzierbarkeitspaket zum finalen Lauf 20260629T202543Z. Beigefügte Abgabe-ZIP Grundprojekt_Code_final.zip, SHA-256: 097bd4ef31a82da5dccf6adb9a2d66744da5d937c6f68baaa1dc4030d017662. Enthält Notebook, Konfiguration, Environment-Snapshot, Daten/Splits, Ergebnisdateien, synthetische Outputs, Rohoutputs/Traces, Zeilenherkunft, Diagnoseauswertungen, Hashes und Manifest (2026)
13. Patki, N., Wedge, R., Veeramachaneni, K.: The Synthetic Data Vault. In: IEEE Int. Conf. on Data Science and Advanced Analytics (DSAA 2016), 399–410 (2016)
14. Esteban, C., Hyland, S.L., Rätsch, G.: Real-valued medical time series generation with recurrent conditional GANs. arXiv preprint arXiv:1706.02633 (2017)
15. Jordon, J., Jarrett, D., Saveliev, E., Yoon, J., Elbers, P., Thorat, P., Ercole, A., Zhang, C., Belgrave, D., van der Schaar, M.: Hide-and-Seek Privacy Challenge: Synthetic Data Generation vs. Patient Re-identification. In: NeurIPS 2020 Competition and Demonstration Track. PMLR 133, 206–215 (2021)
16. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al.: Scikit-learn: machine learning in Python. J. Mach. Learn. Res. 12, 2825–2830 (2011)
17. Dwork, C., Roth, A.: The algorithmic foundations of differential privacy. Foundations and Trends in Theoretical Computer Science 9(3–4), 211–407 (2014)
18. Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., Galstyan, A.: A survey on bias and fairness in machine learning. ACM Comput. Surv. 54(6), 1–35 (2021)

Anmerkung zur KI-Nutzung

ChatGPT unterstützte bei sprachlicher Überarbeitung, Strukturierung, Codeerstellung und Validierungsfragen. Methodenwahl, Experimentdesign, Durchführung, Ergebnisprüfung, Interpretation und Verantwortung für den Inhalt liegen beim Autor.